

Model Conversion

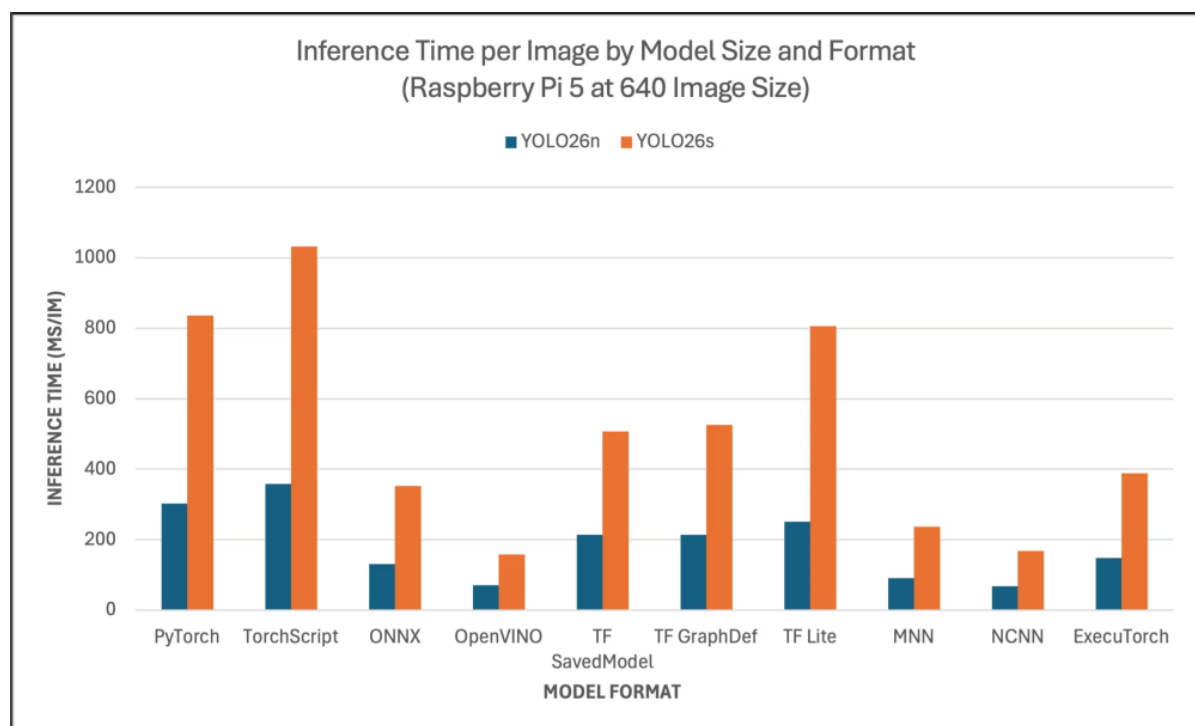
Model Conversion

1. Raspberry Pi 5 YOLO26 (Benchmark)
[Comparison Chart](#)
2. Model Conversion
 - 2.1 CLI: pt → onnx, pt → ncnn
 - 2.2 Python: pt → onnx → ncnn
4. Model Prediction
[CLI Usage](#)
[References](#)

1. Raspberry Pi 5 YOLO26 (Benchmark)

The Ultralytics team has benchmarked YOLO26 on ten different model formats, measuring speed and accuracy. These formats include: PyTorch, TorchScript, ONNX, OpenVINO, TF SavedModel, TF GraphDef, TF Lite, MNN, NCNN, ExecuTorch. The benchmarks were run on Raspberry Pi 5 with FP32 precision and a default input image size of 640.

Comparison Chart



2. Model Conversion

The first time you use YOLO26's export mode, it will automatically install some dependencies. Just wait for the automatic completion!

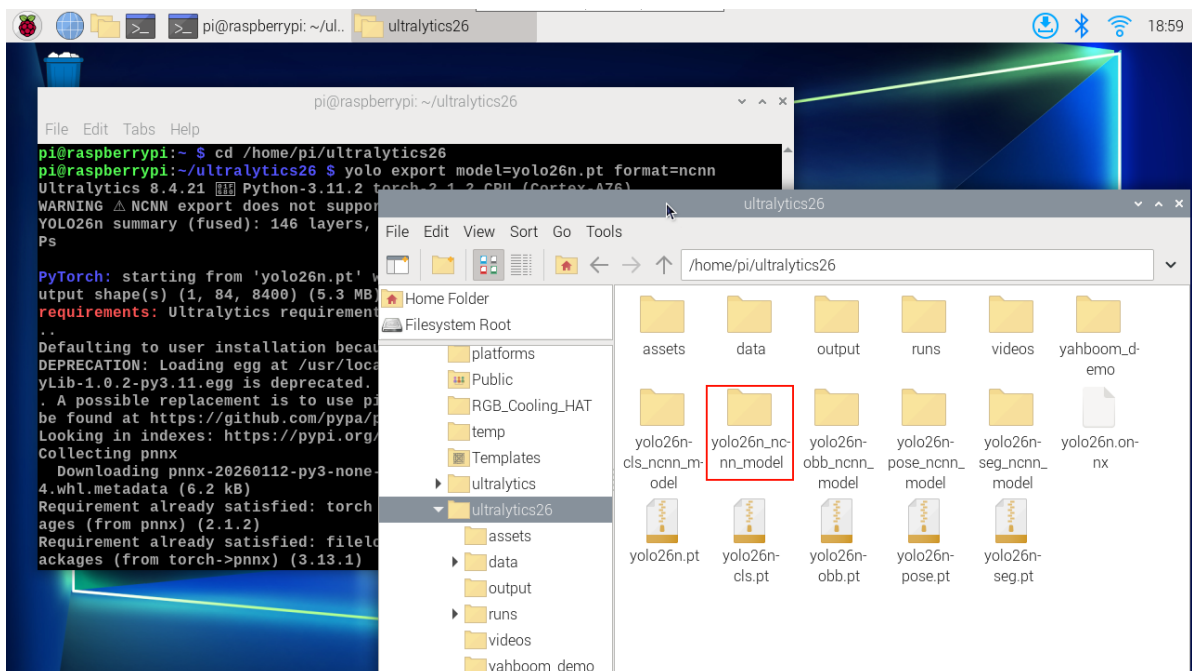
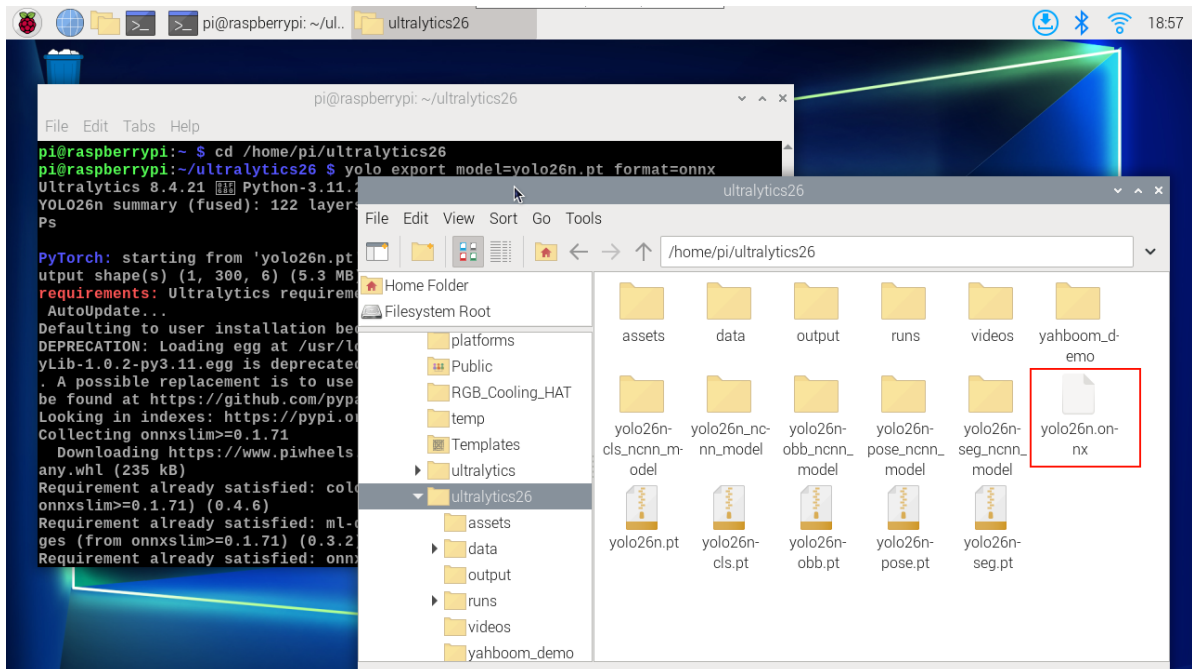
2.1 CLI: pt → onnx, pt → ncnn

Convert PyTorch format models to onnx and ncnn

```
cd /home/pi/ultralytics26
```

```
yolo export model=yolo26n.pt format=onnx
# yolo export model=yolo26n-seg.pt format=onnx
# yolo export model=yolo26n-pose.pt format=onnx
# yolo export model=yolo26n-cls.pt format=onnx
# yolo export model=yolo26n-obb.pt format=onnx
```

```
yolo export model=yolo26n.pt format=ncnn
# yolo export model=yolo26n-seg.pt format=ncnn
# yolo export model=yolo26n-pose.pt format=ncnn
# yolo export model=yolo26n-cls.pt format=ncnn
# yolo export model=yolo26n-obb.pt format=ncnn
```



2.2 Python: pt → onnx → ncnn

Convert PyTorch format models to TensorRT: The conversion process will automatically generate ONNX models

```
cd /home/pi/ultralytics26/yahboom_demo
```

```
python3 model_pt_onnx_ncnn.py
```

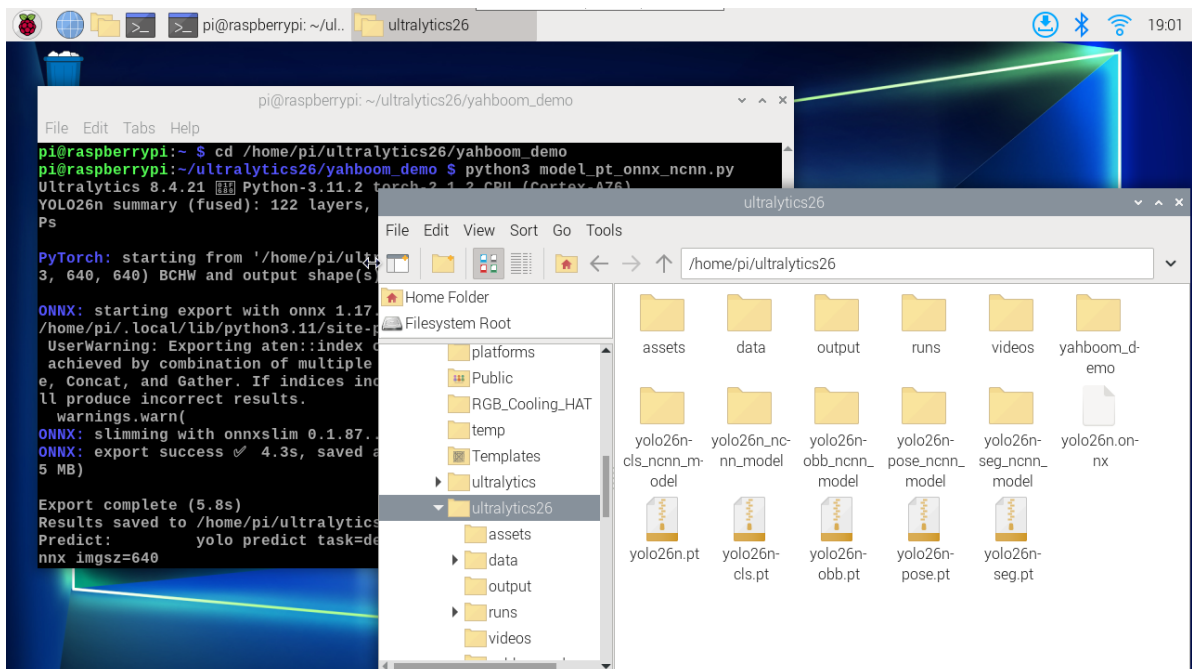
```
from ultralytics import YOLO

# Load a YOLO PyTorch model
model = YOLO("/home/pi/ultralytics26/yolo26n.pt")
# model = YOLO("/home/pi/ultralytics26/yolo26n-seg.pt")
# model = YOLO("/home/pi/ultralytics26/yolo26n-pose.pt")
# model = YOLO("/home/pi/ultralytics26/yolo26n-cls.pt")
# model = YOLO("/home/pi/ultralytics26/yolo26n-obb.pt")

# Export the model to ONNX format
model.export(format="onnx") # This will create 'yolo26n.onnx' in the same
directory

# Export the model to NCNN format
model.export(format="ncnn") # creates 'yolo26n_ncnn_model'
```

Note: The converted model files are located at the converted model file location



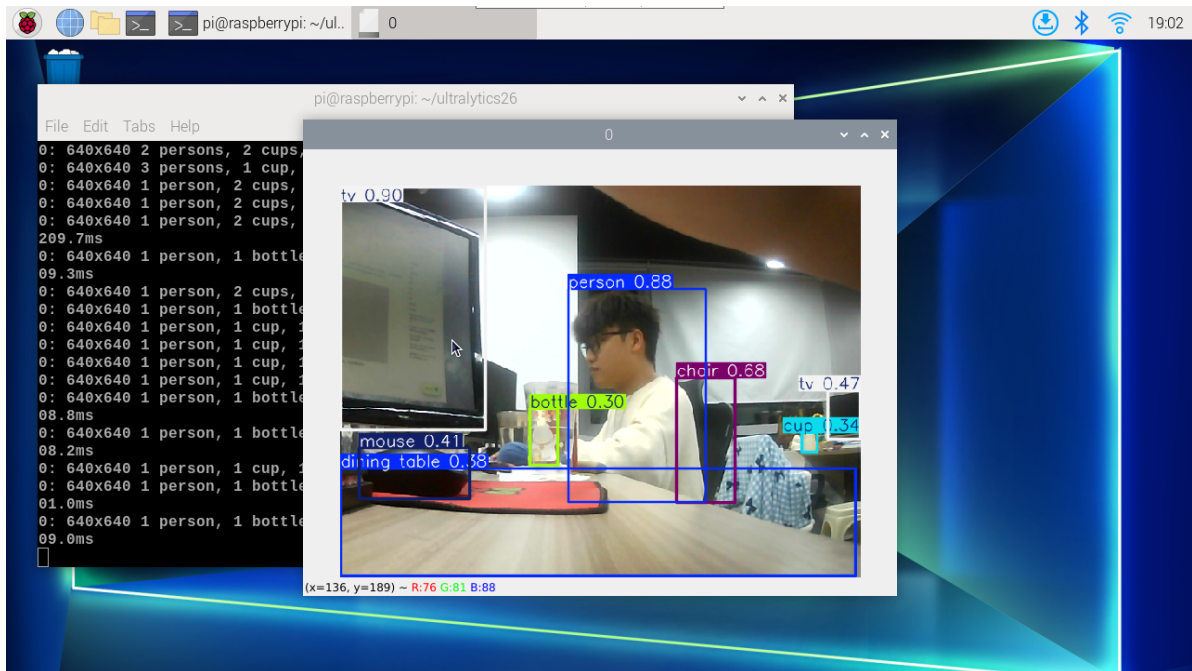
4. Model Prediction

CLI Usage

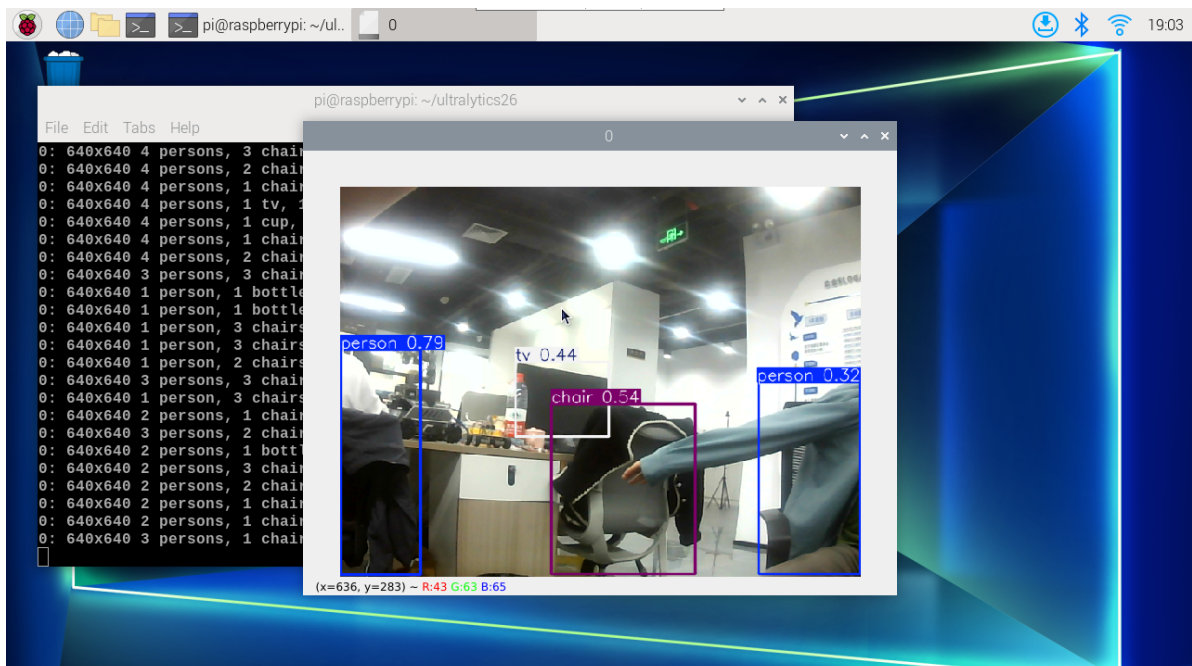
CLI currently only supports calling USB cameras. CSI camera users can directly modify the previous python code to call onnx and ncnn models!

```
cd /home/pi/ultralytics26
```

```
yolo predict model=yolo26n.onnx source=0 save=False show
```



```
yolo predict model=yolo26n_ncnn_model source=0 save=False show
```



References

[Quick Start Guide: Raspberry Pi with Ultralytics YOLOv26](#)

